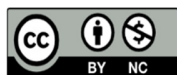


SPRING BOOT



TUTORIAL

Em Português (PT-PT)



Luís Simões da Cunha (2025)

Índice

	Parte 1 – Instalação e Ambiente Backend	11
	Objetivo	11
	1. Instalar o Java (JDK 11 ou superior)	11
	Passos:	11
	Verifica se está instalado:	11
	2. Instalar o Eclipse IDE	11
	Passos:	11
	3. Entender o Maven (gestor de dependências)	12
	4. Criar o Primeiro Projeto com Spring Initializr	12
	Passos:	12
	5. Importar o Projeto no Eclipse	12
	Passos:	12
	6. Executar o Projeto	13
	7. Instalar o MariaDB	13
	Passos (Windows/macOS/Linux):	13
	Checkpoint	13
	Resumo Visual	13
	O que vem a seguir?	14
	Parte 2 – Injeção de Dependências (DI) com Spring Boot	15
	Objetivo	15
	O que é Injeção de Dependências?	15
	Metáfora rápida:	15
	Em termos técnicos:	15
	Tipos de Injeção	15
	Exemplo Prático: Carro e Dono	15
	1. Entidade Owner .java	15
	2. Entidade Car .java com injeção da relação	16
	Criar o Repositório com @Repository	16
	Criar um Serviço com @Service	16
	Controlador com @Autowired (injeção automática)	17
	Boa Prática: Injeção por Construtor	17

	Exemplo de Teste Unitário com DI	17
	Resumo Visual	18
	Dica de Ouro	18
 SOON	O que vem a seguir?	18
	Parte 3 – Persistência com JPA: Entidades e Repositórios	18
	Objetivo	18
	O que é ORM?	19
	Vamos começar com o H2: a base de dados perfeita para testes.....	19
	Adiciona ao pom.xml:	19
	Criar a tua primeira Entidade JPA – Car.java.....	19
	Criar o Repositório – CarRepository.java	20
	Configurar o H2 no application.properties	20
	Aceder à consola H2:.....	20
	Inicializar dados com CommandLineRunner	21
	Verifica no browser.....	21
	Resumo Visual	21
 SOON	O que vem a seguir?	22
	Parte 4 – Relacionamentos entre Tabelas e Repositórios Avançados	23
	Objetivos	23
	Exemplo Real: Donos e Carros	23
	Passo 1: Criar a entidade Owner	23
	Passo 2: Criar o OwnerRepository	24
	Passo 3: Ligar o Car ao Owner com @ManyToOne	24
	Passo 4: Popular a base de dados com relações	24
	Passo 5: Consultas personalizadas com @Query	24
	Exemplos com palavras-chave	25
	Repositórios com Paginação e Ordenação	25
	Resumo Visual	25
	Dica de Ouro	25
 SOON	O que vem a seguir?	25
	Parte 5 – Criar Web Services RESTful com Spring Boot	26
	Objetivo	26

🤖 O que é uma API RESTful?	26
🧠 Princípios básicos:.....	26
🔧 Criar um Controlador REST com Spring Boot	26
📄 CarController.java.....	26
⚡ Alternativa rápida: Spring Data REST	27
🧪 Testar a API com Postman	27
📦 O que é o Postman?	27
🧪 Exemplos de testes:.....	27
🧠 Resumo Visual	28
🔥 Dica de Ouro	28
➡️ O que vem a seguir?	28
🔒 Parte 6 – Segurança e Testes no Backend	29
🎯 Objetivo	29
🧠 O que é Spring Security?	29
🧱 Blocos essenciais da segurança:.....	29
🔒 Exemplo simples: autenticação com JWT.....	29
1. Adicionar dependências no pom.xml	29
2. Criar uma classe User e Role.....	30
3. Criar uma classe que implementa UserDetailsService.....	30
4. Configurar o SecurityFilterChain.....	30
5. Gerar o token JWT ao fazer login.....	31
🛡️ Exemplo de classe utilitária para JWT	31
🧪 Testes Unitários com JUnit 5	32
Testar lógica de negócio:	32
Testar controladores com @WebMvcTest	32
🧠 Resumo Visual	32
🔒 Dica de Ouro	33
➡️ O que vem a seguir?	33
⚙️ Parte 7 – Ambiente de Desenvolvimento React	34
🎯 Objetivo	34
🧠 O que é o Node.js?	34
💻 Instalar o Visual Studio Code (VS Code)	34
🔧 Criar um Projeto com create-react-app.....	34

👉 Abre o terminal e escreve:	34
▶ Executar a App	35
No terminal:	35
✏ Modificar a App Base	35
🧠 Resumo Visual	35
💡 Dica de Ouro	36
➡ O que vem a seguir?	36
🔗 Parte 8 – Fundamentos de React: Componentes e JSX.....	37
🎯 Objetivos	37
🧱 1. O que são Componentes?	37
🧠 Dica:	37
🧩 2. JSX: O “HTML do React”	37
Regras de JSX:	37
✉ 3. Props: Dados que viajam até ao componente	37
🧠 4. State: Quando o componente precisa de memória.....	38
🔄 5. Renderização Condicional.....	38
👤 6. Componentes Stateless vs Stateful.....	38
Exemplo stateless:	38
🧠 Resumo Visual	39
💡 Dica de Ouro	39
➡ O que vem a seguir?	39
🧠 Parte 9 – Hooks e Context API: React Moderno	40
🎯 Objetivos desta parte	40
🔄 1. useState: guardar estado num componente	40
🕒 2. useEffect: efeitos colaterais (ex: chamadas API)	40
🔍 3. useRef: referência a elementos ou valores mutáveis	41
🛠 4. Criar um hook personalizado	41
🌐 5. Context API: partilhar estado global	42
Criar e usar Contexto:	42
🧠 Resumo Visual	42
💡 Dica de Ouro	43
➡ A seguir.....	43

🧩	Parte 10 – Eventos, Listas e Formulários em React	44
🎯	Objetivos	44
🎮	1. Gestão de Eventos	44
📄	2. Mapeamento de Listas	44
✏️	3. Formulários Controlados	45
⚙️	4. Formulários Não Controlados.....	45
🧠	Resumo Visual	46
💡	Dica de Ouro	46
➡️ SOON	A seguir.....	46
🔗	Parte 11 – Consumir APIs REST com React	47
🎯	Objetivos desta parte	47
🌐	1. Como funciona uma chamada API?	47
🔍	2. Usar fetch (nativo do browser)	47
⚙️	3. Usar axios (mais poderoso).....	48
🧪	4. Promessas e Assíncrono	48
💥	5. POST, PUT e DELETE com axios.....	49
	Criar (POST):	49
	Atualizar (PUT):	49
	Apagar (DELETE):.....	49
🧠	Resumo Visual	49
💡	Dica de Ouro	49
➡️ SOON	A seguir.....	49
🎨	Parte 12 – Componentes de Terceiros e MUI	49
🎯	Objetivos	49
📦	1. AG Grid – Tabelas dinâmicas e poderosas	50
📦	Instalar:.....	50
🧪	Exemplo de tabela:	50
🧱	2. Material UI – Interface com estilo	50
📦	Instalar:.....	50
🔵	Botões e campos:	50
💡	Ícones:.....	51
🌀	3. Navegação com React Router	51

 Instalar:.....	51
 Configurar rotas:.....	51
 Resumo Visual	51
 Dica de Ouro	51
 A seguir.....	52
 Parte 13 – Integração Completa: Frontend para o Backend	53
 Objetivos desta parte	53
 1. Estrutura recomendada da aplicação integrada.....	53
 2. Mockup da interface (Ex: Lista de Carros)	53
 3. Ligação à API Spring Boot	54
 4. Adicionar Navegação com React Router.....	54
 Resumo Visual	55
 Dica de Ouro	55
 A seguir.....	55
 Parte 14 – CRUD Completo com React + Spring Boot.....	56
 Objetivos desta parte	56
 Estrutura Recomendada	56
 1. Listar Registos (Read).....	56
 2. Adicionar Registos (Create)	57
 3. Atualizar Registos (Update)	58
 4. Serviço API (CarroService.js).....	58
 5. Feedback Visual	58
 Resumo CRUD	58
 Dica de Ouro	59
 A seguir.....	59
 Parte 15 – Estilização Profissional com MUI	60
 Objetivos	60
 1. Instalação do Material UI.....	60
 2. Usar Botões com Estilo	60
 3. Formulários com TextField.....	60
 4. Criar um Tema Personalizado	61
 5. Feedback com Snackbar (avisos rápidos).....	61

🔗 6. Ícones com @mui/icons-material.....	61
💡 Boas Práticas de UX	61
💡 Dica de Ouro	62
➡️ A seguir.....	62
🔗 Parte 16 – Testes Automatizados no Frontend com Jest	63
🎯 Objetivos desta parte	63
⚙️ 1. Instalação do ambiente de testes	63
🔍 2. Testes de componentes básicos.....	63
✅ Teste simples:	63
🖱️ 3. Testar eventos (ex: cliques)	63
✅ Teste de clique:	64
🧩 4. Snapshot Testing	64
🔄 5. Executar os testes	64
📊 Resumo Visual	64
💡 Dica de Ouro	64
➡️ A seguir.....	65
🔒 Parte 17 – Segurança Full Stack com JWT	66
🎯 Objetivos desta parte	66
🛡️ 1. Backend Seguro com Spring Security e JWT	66
🔧 a) Adicionar dependências no pom.xml.....	66
🔒 b) Criar classes essenciais	66
🧠 c) Exemplo: Gerar token após login	66
🔒 d) Proteger endpoints REST.....	67
⚙️ 2. Frontend: Gestão do Token com React.....	67
📦 a) Guardar token no localStorage	67
🔒 b) Enviar token no cabeçalho com axios	67
🔒 c) Proteger rotas com um wrapper de autenticação	67
🌟 d) Exemplo de formulário de login.....	68
📊 Resumo Visual	68
💡 Dica de Ouro	68
➡️ A seguir.....	68
🚀 Parte 18 – Deploy Profissional com Docker e Heroku	70

	Objetivos	70
	1. Criar Dockerfile para o Backend.....	70
	Dockerfile:	70
	Construir a imagem:.....	70
	2. Criar Dockerfile para o Frontend (React).....	70
	Dockerfile:	70
	Construir a imagem:.....	70
	3. Testar com docker - compose	71
	Correr tudo:.....	71
	4. Deploy na Cloud	71
	a) Backend no Heroku (alternativa: Render)	71
	5. Variáveis de Ambiente com .env	72
	6. Segurança e Produção	72
	Dica de Ouro	72
	A seguir.....	72
	Parte 19 – Melhores Práticas para Devs Full Stack.....	73
	Objetivos	73
	1. Versionamento com Git.....	73
	Boas práticas:	73
	Exemplo de fluxo:	73
	2. Boas práticas para APIs REST	73
	3. Convenções de código (backend e frontend)	74
	Java (Spring Boot):	74
	JavaScript / React:	74
	4. Design Responsivo e Mobile-First	74
	Boas práticas:	74
	5. Mentalidade de Profissional Full Stack	74
	Resumo Visual	74
	Dica de Ouro	75
	A seguir.....	75
	Parte 20 – Conclusão e Caminhos para o Futuro	76
	Objetivos	76
	1. Tecnologias Complementares para o Stack	76

🧠 2. GraphQL – A evolução das APIs	76
🔧 3. Testes End-to-End (E2E)	77
🔧 Ferramentas recomendadas:.....	77
Exemplo com Cypress:	77
🔄 4. Integração Contínua e Deploy Contínuo (CI/CD)	77
Exemplos de ferramentas:	77
Exemplo de GitHub Actions para React:	77
📖 5. Roadmap Full Stack – O Teu Próximo Nível	78
🧱 Fundamentos Sólidos	78
🚧 Backend Avançado	78
🧠 Frontend Avançado	78
☁️ DevOps	78
👛 Soft Skills e Carreira	78
💡 Dica Final	78

Parte 1 – Instalação e Ambiente Backend

Começa aqui a tua jornada como desenvolvedor Full Stack! Vamos montar as ferramentas do teu "laboratório digital".

Objetivo

Neste capítulo, vais:

- Instalar o Java (JDK)
 - Configurar o Eclipse IDE
 - Entender o que é o Maven
 - Criar o teu primeiro projeto Spring Boot com o **Spring Initializr**
 - Instalar o MariaDB e preparar o ambiente para persistência de dados
-

1. Instalar o Java (JDK 11 ou superior)

Spring Boot é feito em Java. Por isso, precisas do **JDK – Java Development Kit**.

 Versão recomendada: **JDK 17**

Passos:

- Vai ao site oficial: <https://adoptium.net>
- Escolhe a versão **Temurin 17** (OpenJDK) para o teu sistema operativo (Windows, macOS ou Linux)
- Instala como qualquer programa habitual

Verifica se está instalado:

```
java -version
```

Deverás ver algo como: `openjdk version "17.0.x"`

2. Instalar o Eclipse IDE

O **Eclipse** é como o cockpit onde vais comandar os teus projetos Java.

Passos:

- Vai a: <https://www.eclipse.org/downloads/>
 - Clica em **Download 64-bit**
 - Abre o instalador e escolhe: **Eclipse IDE for Enterprise Java and Web Developers**
 - Segue os passos de instalação
-

3. Entender o Maven (gestor de dependências)

O **Maven** é como uma agenda que organiza tudo o que o teu projeto precisa: bibliotecas, plugins e versões.

💡 Não precisas de instalar o Maven à parte – o Eclipse já vem com Maven embutido.

✨ O ficheiro principal do Maven é o `pom.xml`. Lá dirás: "Quero usar Spring Web!", e o Maven vai buscar tudo o que precisas para isso.

4. Criar o Primeiro Projeto com Spring Initializr

O **Spring Initializr** é como um gerador de arranque rápido 🚀.

 Passos:

1. Vai a: <https://start.spring.io>
2. Preenche o formulário:
 - **Project:** Maven
 - **Language:** Java
 - **Spring Boot:** 2.7.x ou superior (evita a 3.x para já)
 - **Group:** com.exemplo
 - **Artifact:** cardatabase
 - **Name:** cardatabase
 - **Java version:** 17
3. Clique em "**Add Dependencies**" e adiciona:
 - **Spring Web**
 - **Spring Boot DevTools**
4. Clica em **Generate** para obteres um `.zip`
5. Extraí o `.zip` para uma pasta à tua escolha

5. Importar o Projeto no Eclipse

 Passos:

1. Abre o Eclipse
2. Vai a **File > Import**
3. Escolhe **Existing Maven Projects**
4. Selecciona a pasta onde extraíste o `.zip`
5. Clica em **Finish**

🌈 Deverás ver o projeto cardatabase no lado esquerdo (Project Explorer)

6. Executar o Projeto

1. Abre a classe `CardatabaseApplication.java`
2. Clica com o botão direito e escolhe **Run As > Java Application**

Se tudo correr bem, no **Console** verás algo como:

Started CardatabaseApplication in X seconds

7. Instalar o MariaDB

Vamos precisar de uma base de dados! O **MariaDB** é leve, rápido e open-source — ideal para começarmos!

Passos (Windows/macOS/Linux):

- Vai a: <https://mariadb.org/download/>
- Faz o download do **MariaDB Community Server**
- Durante a instalação:
 - Define uma **senha para o utilizador root**
 - Podes deixar as restantes opções por defeito
 - Instala também o cliente gráfico **HeidiSQL** (opcional, mas recomendado)

✶ Guarda bem a tua palavra-passe do root – vamos usá-la mais tarde.

Checkpoint

Antes de avançarmos, verifica se:

- Consegues correr o projeto Spring Boot a partir do Eclipse
 - O ficheiro `pom.xml` já tem as dependências que escolheste
 - Instalaste o MariaDB e sabes a password do root
-

Resumo Visual

Ferramenta	Para quê serve?
Java (JDK)	Para programar em Java
Eclipse	Para editar e correr os projetos
Maven	Para gerir dependências
Spring Boot	Para criar aplicações web rápidas
MariaDB	Para guardar dados (base de dados)
Spring Initializr	Para arrancar um projeto em minutos

O que vem a seguir?

👉 Na **Parte 2**, vamos explorar **Injeção de Dependências**, uma das grandes magias do Spring Boot que permite ligar todas as peças do puzzle com facilidade e testabilidade.

⚙️ Parte 2 – Injeção de Dependências (DI) com Spring Boot

Imagina uma orquestra onde cada instrumento entra no momento certo... sem maestro? Impossível. O Spring Boot é o maestro — e a **injeção de dependências (DI)** é a sua batuta.

🎯 Objetivo

Neste capítulo, vais:

- Compreender o que é **Injeção de Dependências (DI)**
- Usar as anotações `@Autowired`, `@Service` e `@Repository`
- Criar um exemplo simples e funcional
- Aprender boas práticas de DI no Spring Boot

🧠 O que é Injeção de Dependências?

📖 Metáfora rápida:

Pensa numa **máquina de café** ☕ :

- O **Cliente (Carro)** precisa de **café (Dono)**
- A **máquina (Injector)** fornece o café — o **cliente não tem de saber onde está o café nem como foi feito!**

🎯 Em termos técnicos:

DI é um padrão onde os objetos (dependências) são **fornecidos automaticamente** a uma classe, em vez de esta os criar diretamente.

⚙️ Tipos de Injeção

Tipo	Exemplo
🔧 Constructor	via o construtor da classe
📦 Setter	via um método <code>setX(...)</code>
⚡ Campo (<code>@Autowired</code>)	direto no atributo

🌟 Exemplo Prático: Carro e Dono

1. Entidade `Owner.java`

```
@Entity
public class Owner {
    @Id @GeneratedValue
    private Long id;
```

```
private String nome;

// Construtores, getters e setters
}
```

2. Entidade Car.java com injeção da relação

```
@Entity
public class Car {
    @Id @GeneratedValue
    private Long id;
    private String modelo;

    @ManyToOne
    private Owner owner;

    // Construtores, getters e setters
}
```

Criar o Repositório com @Repository

```
@Repository
public interface CarRepository extends CrudRepository<Car, Long> {
    List<Car> findByModelo(String modelo);
}
```

 Spring **detecta automaticamente** esta classe como componente e trata da criação da instância!

Criar um Serviço com @Service

```
@Service
public class CarService {

    private final CarRepository carRepo;

    // Injeção via construtor (boa prática!)
    public CarService(CarRepository carRepo) {
        this.carRepo = carRepo;
    }

    public List<Car> getAllCars() {
        return (List<Car>) carRepo.findAll();
    }
}
```

Controlador com @Autowired (injeção automática)

```
@RestController
@RequestMapping("/api/cars")
public class CarController {

    private final CarService carService;

    // Injeção via construtor (recomendado)
    public CarController(CarService carService) {
        this.carService = carService;
    }

    @GetMapping
    public List<Car> listar() {
        return carService.getAllCars();
    }
}
```

Boa Prática: Injeção por Construtor

Evita usar @Autowired diretamente no campo. Usa construtores:

```
public class Exemplo {
    private final Dependencia dep;

    public Exemplo(Dependencia dep) {
        this.dep = dep;
    }
}
```

Porquê?

- Facilita testes unitários
 - Garante imutabilidade
 - Deteta erros mais cedo
-

Exemplo de Teste Unitário com DI

```
@Test
void deveListarCarros() {
    CarRepository repo = Mockito.mock(CarRepository.class);
    CarService servico = new CarService(repo);

    // Simular dados
    Mockito.when(repo.findAll()).thenReturn(List.of(new Car("Tesla")));

    List<Car> resultado = servico.getAllCars();
}
```

```
Assertions.assertEquals(1, resultado.size());  
}
```

Resumo Visual

Anotação	Serve para...
@Autowired	Injetar dependências automaticamente
@Service	Classe de lógica de negócio
@Repository	Acesso à base de dados
@Component	Componente genérico que o Spring pode injetar
@RestController	Controlador de API REST

Dica de Ouro

“**Injeção de dependências é como magia — quando bem usada, torna tudo invisivelmente mais simples.** Mas cuidado: magia mal usada dá sarilho! ✨🐉”

O que vem a seguir?

Na **Parte 3**, vais aprender como criar e aceder a **bases de dados com JPA e Hibernate**, incluindo relações e repositórios reais.

Parte 3 – Persistência com JPA: Entidades e Repositórios

Chegou o momento de dar memória à tua aplicação! Com JPA, H2 e Spring Boot, vais ensinar o teu programa a guardar e a recuperar informação como um verdadeiro gestor de bases de dados.

Objetivo

Neste capítulo vais:

- Entender o que é **ORM (Object-Relational Mapping)**
 - Criar **entidades JPA** com Spring Boot
 - Usar uma base de dados em memória **H2**
 - Configurar propriedades de persistência
 - Criar repositórios com **CRUD automático**
-

O que é ORM?

Object-Relational Mapping (ORM) é como uma ponte entre o mundo dos objetos (Java) e o mundo relacional (tabelas de bases de dados).

 **Metáfora visual:** Imagina uma **folha de cálculo Excel** onde cada linha é um objeto Java (ex: Carro) e cada coluna é um campo (ex: marca, preço). O ORM trata da tradução entre o que tens no código e o que está gravado na "folha de Excel".

Vamos começar com o H2: a base de dados perfeita para testes

O **H2** é uma base de dados em memória:

-  Super-rápida
-  Ideal para testes
-  Limpa-se automaticamente a cada arranque

 Adiciona ao pom.xml:

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

Criar a tua primeira Entidade JPA – Car.java

```
@Entity
public class Car {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    private String brand;
    private String model;
    private String color;
    private int year;
    private int price;

    // Construtor vazio (obrigatório para JPA)
    public Car() {}

    public Car(String brand, String model, String color, int year, int price) {
```

```

        this.brand = brand;
        this.model = model;
        this.color = color;
        this.year = year;
        this.price = price;
    }

    // Getters e setters...
}

```

🎓 A anotação `@Entity` diz ao Spring: "isto é uma tabela!"

📌 `@Id` + `@GeneratedValue` indica que o campo é a **chave primária autogerada**

📁 Criar o Repositório – `CarRepository.java`

```

@Repository
public interface CarRepository extends CrudRepository<Car, Long> {
    List<Car> findByBrand(String brand);
    List<Car> findByColor(String color);
}

```

📦 `CrudRepository` já vem com métodos como `save`, `findAll`, `deleteById`, etc.

🎯 E podes definir os teus métodos com nomes descritivos, como `findByBrand` – e o Spring **cria a query por ti!**

⚙️ Configurar o H2 no `application.properties`

```

spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

spring.h2.console.enabled=true
spring.jpa.show-sql=true

```

Aceder à consola H2:

🖥️ Abre o browser e vai para:

`http://localhost:8080/h2-console`

- JDBC URL: `jdbc:h2:mem:testdb`
 - Username: `sa`
 - Password: (deixa vazio)
-

Inicializar dados com CommandLineRunner

Na CardatabaseApplication.java:

```
@SpringBootApplication
public class CardatabaseApplication implements CommandLineRunner {

    @Autowired
    private CarRepository repository;

    public static void main(String[] args) {
        SpringApplication.run(CardatabaseApplication.class, args);
    }

    @Override
    public void run(String... args) throws Exception {
        repository.save(new Car("Toyota", "Corolla", "Branco", 2020, 18000));
        repository.save(new Car("Ford", "Mustang", "Vermelho", 2021, 35000));
        repository.findAll().forEach(car ->
            System.out.println(car.getBrand() + " " + car.getModel())
        );
    }
}
```

Verifica no browser

Depois de correr a app:

1. Acede ao <http://localhost:8080/h2-console>
2. Liga-te com jdbc:h2:mem:testdb
3. Corre esta query:

```
SELECT * FROM CAR;
```

💡 Vais ver os teus carros!

Resumo Visual

Elemento	Anotação	Para quê?
Entidade	@Entity	Indica que é uma tabela
Chave primária	@Id + @GeneratedValue	Define a coluna id como PK
Repositório	CrudRepository<T, ID>	Permite guardar e buscar objetos
Query automática	findByX	O Spring cria a query por ti

O que vem a seguir?

Na **Parte 4**, vais aprender a:

- Adicionar **relações entre entidades** (OneToMany, ManyToOne)
- Usar repositórios ainda mais poderosos
- Criar queries personalizadas com @Query

Parte 4 – Relacionamentos entre Tabelas e Repositórios Avançados

Chegou o momento de criar ligações entre as entidades da tua aplicação — porque, no mundo real, as coisas estão ligadas umas às outras!

Objetivos

- Criar **relações entre entidades** com JPA: `@OneToMany` e `@ManyToOne`
 - Compreender o uso de **chaves estrangeiras (foreign keys)**
 - Usar **consultas personalizadas** com `@Query`
 - Aprender boas práticas com `FetchType.LAZY`
-

Exemplo Real: Donos e Carros

 Um **dono** pode ter **vários carros**



Mas cada **carro** tem apenas **um dono**

Passo 1: Criar a entidade `Owner`

`@Entity`

```
public class Owner {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private long ownerid;

    private String firstname;
    private String lastname;

    public Owner() {}

    public Owner(String firstname, String lastname) {
        this.firstname = firstname;
        this.lastname = lastname;
    }

    // Getters e Setters
}
```

Passo 2: Criar o OwnerRepository

```
public interface OwnerRepository extends CrudRepository<Owner, Long> {}
```

Este repositório vai permitir guardar e pesquisar donos.

Passo 3: Ligar o Car ao Owner com @ManyToOne

Na classe Car:

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "owner") // Nome da FK na base de dados
private Owner owner;

// Getters e setters
```

Boas práticas:

Usar FetchType.LAZY evita carregar dados desnecessários (ideal para performance!)

Passo 4: Popular a base de dados com relações

No CardatabaseApplication.java:

```
@Autowired
private CarRepository carRepo;

@Autowired
private OwnerRepository ownerRepo;

@Override
public void run(String... args) throws Exception {
    Owner owner1 = new Owner("João", "Silva");
    Owner owner2 = new Owner("Ana", "Costa");
    ownerRepo.saveAll(List.of(owner1, owner2));

    carRepo.save(new Car("Toyota", "Corolla", "Branco", 2020, 18000, owner1));
    carRepo.save(new Car("Ford", "Mustang", "Vermelho", 2021, 35000, owner2));
}
```

 Tens de ajustar o construtor da classe Car para aceitar um Owner como último parâmetro.

Passo 5: Consultas personalizadas com @Query

Cria queries mais flexíveis com a anotação @Query:

```
@Query("select c from Car c where c.brand = ?1")
List<Car> findByBrand(String brand);
```



```
@Query("select c from Car c where c.owner.lastname = ?1")
List<Car> findByOwnerLastname(String lastname);
```

💡 Estas queries usam **JPQL**, uma espécie de “SQL orientado a objetos”.

Exemplos com palavras-chave

O Spring cria queries com base no nome do método!

```
List<Car> findByBrandAndModel(String brand, String model);
List<Car> findByColorOrYear(String color, int year);
List<Car> findByBrandOrderByYearAsc(String brand);
```

Repositórios com Paginação e Ordenação

Para grandes volumes de dados:

```
public interface CarRepository extends PagingAndSortingRepository<Car, Long> {}
```

Permite:

```
Page<Car> findAll(Pageable pageable);
```

Resumo Visual

Relação	Anotação	Local
Muitos para Um	@ManyToOne	na classe "filha" (ex: Car)
Um para Muitos	@OneToMany	opcional, na classe "pai" (ex: Owner)
Foreign Key	@JoinColumn	define a FK na base de dados
Query personalizada	@Query(...)	no repositório JPA

Dica de Ouro

“Relacionamentos em JPA são como amizades: fáceis de criar, difíceis de gerir... se não forem bem pensadas! Usa sempre LAZY para evitar surpresas de performance. 🧠”

SOON O que vem a seguir?

Na **Parte 5**, vamos criar os nossos **primeiros Web Services RESTful**, abrindo a porta da aplicação ao mundo exterior! 🌍 📡

Parte 5 – Criar Web Services RESTful com Spring Boot

Chegou o momento de transformar a tua aplicação num serviço online! Vamos criar endpoints RESTful e testar tudo com o Postman.

Objetivo

Nesta parte vais:

- Compreender o que é uma **API RESTful**
 - Criar **controladores REST** com Spring Boot
 - Expor endpoints para listar, adicionar e apagar dados
 - Usar o **Spring Data REST** para gerar endpoints automaticamente
 - Testar a API com o **Postman**
-

O que é uma API RESTful?

REST é uma arquitetura de comunicação entre aplicações web. O objetivo é **expor recursos** (ex: carros, utilizadores) através de URLs, como se fossem páginas web — mas para sistemas, não para pessoas.

Princípios básicos:

Verbo HTTP	Ação	Exemplo
GET	Ler dados	/api/cars
POST	Criar dados	/api/cars
PUT	Atualizar	/api/cars/1
DELETE	Apagar	/api/cars/1

Criar um Controlador REST com Spring Boot

 CarController.java

```
@RestController
@RequestMapping("/api/cars")
public class CarController {

    private final CarRepository carRepo;

    public CarController(CarRepository carRepo) {
        this.carRepo = carRepo;
    }

    @GetMapping
    public Iterable<Car> listarTodos() {
        return carRepo.findAll();
    }
}
```

```

    }

    @PostMapping
    public Car adicionar(@RequestBody Car novoCarro) {
        return carRepo.save(novoCarro);
    }

    @DeleteMapping("/{id}")
    public void apagar(@PathVariable Long id) {
        carRepo.deleteById(id);
    }
}

💡 @RestController junta @Controller + @ResponseBody → devolve JSON!
🔗 @RequestMapping("/api/cars") define a rota base
🎯 @RequestBody transforma o JSON num objeto Java
🕒 @PathVariable lê o ID da URL

```

⚡ Alternativa rápida: Spring Data REST

Queres endpoints automáticos para o repositório? Basta adicionares esta dependência ao `pom.xml`:

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-rest</artifactId>
</dependency>

```

E pronto! Sem escrever controladores, tens um serviço REST funcional em:

`http://localhost:8080/api/cars`

💡 Podes manter esta opção para protótipos rápidos, mas para produção recomenda-se criar controladores personalizados (como o exemplo acima) para ter mais controlo.

🔧 Testar a API com Postman

📦 O que é o Postman?

Uma ferramenta para testar APIs REST — envia pedidos HTTP e mostra as respostas.

🔧 Exemplos de testes:

📋 *GET /api/cars*

- Método: GET
- Verifica se a lista de carros aparece

+ *POST /api/cars*

- Método: POST
- Headers: Content-Type: application/json

- Body (raw, JSON):

```
{
  "brand": "Tesla",
  "model": "Model S",
  "color": "Preto",
  "year": 2022,
  "price": 75000
}
```

 *DELETE /api/cars/1*

- Método: DELETE
- Remove o carro com id=1

Resumo Visual

Conceito	Anotação / Ferramenta	Função
Controlador REST	@RestController	Expõe métodos como endpoints
Rota base	@RequestMapping("/api/...")	Define prefixo das URLs
Parâmetros URL	@PathVariable	Lê dados da URL (ex: ID)
Corpo do pedido	@RequestBody	Converte JSON → objeto Java
Spring Data REST	spring-boot-starter-data-rest	Gera API automaticamente
Postman	Ferramenta externa	Testar endpoints

Dica de Ouro

“RESTful é como um restaurante: cada pedido tem um verbo, um caminho e um resultado. GET é o menu, POST é fazer o pedido, DELETE é cancelar! 🍽️ 🚀”

O que vem a seguir?

Na **Parte 6**, vamos mergulhar no mundo da **segurança com Spring Security**, onde aprenderás a proteger a tua API com autenticação, JWTs e autorização! 🛡️ 🌐

Parte 6 – Segurança e Testes no Backend

Proteger a tua aplicação é tão importante como construí-la. E testar é como fazer o check-up antes de a lançares ao mundo!

Objetivo

Nesta parte vais aprender a:

- Proteger endpoints com **Spring Security**
- Implementar autenticação com **JWT (JSON Web Tokens)**
- Criar utilizadores com **roles**
- Escrever **testes unitários com JUnit**

O que é Spring Security?

É uma **estrutura poderosa de autenticação e autorização** que protege endpoints e gere sessões, roles e tokens.

Blocos essenciais da segurança:

Componente	Função
SecurityFilterChain	Define as regras de segurança (quem acede ao quê)
UserDetailsService	Como são carregados os utilizadores
PasswordEncoder	Como as passwords são guardadas
JWT	Como os tokens são gerados e verificados

Exemplo simples: autenticação com JWT

1. Adicionar dependências no pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt</artifactId>
  <version>0.9.1</version>
</dependency>
```

2. Criar uma classe User e Role

```
@Entity
public class AppUser {
    @Id
    @GeneratedValue
    private Long id;
    private String username;
    private String password;

    @ManyToMany(fetch = FetchType.EAGER)
    private Set<AppRole> roles;
}
```

```
@Entity
public class AppRole {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
}
```

3. Criar uma classe que implementa UserDetailsService

```
@Service
public class MyUserDetailsService implements UserDetailsService {
    @Autowired
    private UserRepository userRepo;

    @Override
    public UserDetails loadUserByUsername(String username) {
        AppUser user = userRepo.findByUsername(username);
        return new User(user.getUsername(), user.getPassword(),
            user.getRoles().stream()
                .map(r -> new SimpleGrantedAuthority(r.getName()))
                .toList());
    }
}
```

4. Configurar o SecurityFilterChain

```
@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf().disable()
            .authorizeRequests()
                .antMatchers("/api/auth/**").permitAll()
                .anyRequest().authenticated()
            .and()
    }
}
```

```

        .sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS);

        http.addFilterBefore(new JwtFilter(),
UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}

```

5. Gerar o token JWT ao fazer login

```

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    private AuthenticationManager authManager;

    @Autowired
    private JwtUtil jwtUtil;

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest req) {
        Authentication auth = authManager.authenticate(
            new UsernamePasswordAuthenticationToken(req.getUsername(), req.getPassword())
        );
        String token = jwtUtil.generateToken(req.getUsername());
        return ResponseEntity.ok(new JwtResponse(token));
    }
}

```

Exemplo de classe utilitária para JWT

```

@Component
public class JwtUtil {
    private String secret = "segredo123";

    public String generateToken(String username) {
        return Jwts.builder()
            .setSubject(username)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + 86400000)) // 1
dia
            .signWith(SignatureAlgorithm.HS256, secret)
            .compact();
    }

    public String extractUsername(String token) {
        return Jwts.parser().setSigningKey(secret)

```

```

        .parseClaimsJws(token)
        .getBody().getSubject();
    }
}

```

Testes Unitários com JUnit 5

Testar lógica de negócio:

```

@SpringBootTest
public class CarServiceTest {

    @MockBean
    private CarRepository carRepo;

    @Autowired
    private CarService carService;

    @Test
    public void testGetAllCars() {
        List<Car> cars = List.of(new Car("Tesla", "Model S", "Preto", 2022, 80000));
        Mockito.when(carRepo.findAll()).thenReturn(cars);

        List<Car> result = carService.getAllCars();
        Assertions.assertEquals(1, result.size());
    }
}

```

Testar controladores com @WebMvcTest

```

@WebMvcTest(CarController.class)
public class CarControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private CarService carService;

    @Test
    public void testListCars() throws Exception {
        mockMvc.perform(get("/api/cars"))
            .andExpect(status().isOk());
    }
}

```

Resumo Visual

Conceito	Ferramenta / Anotação
----------	-----------------------

Conceito	Ferramenta / Anotação
Autenticação e Autorização	@EnableWebSecurity, JWT, Roles
Injeção de utilizadores	UserDetailsService, @Autowired
Geração de Token	JwtUtil
Testes de lógica	@SpringBootTest, @MockBean
Testes de endpoints	@WebMvcTest, MockMvc

Dica de Ouro

“Um sistema sem testes é como uma casa sem alicerces. E sem segurança... é uma porta aberta!

 ”

O que vem a seguir?

Na **Parte 7**, vais configurar o **ambiente de frontend com React**, para poderes comunicar com os endpoints que acabaste de proteger!  

Parte 7 – Ambiente de Desenvolvimento React

Vamos montar a oficina onde o teu frontend React vai ganhar vida! Um ambiente bem configurado é meio caminho andado para um desenvolvimento rápido e agradável.

Objetivo

Nesta parte vais aprender a:

- Instalar o **Node.js** (motor do React)
 - Instalar o **Visual Studio Code**
 - Criar o teu primeiro projeto com `create-react-app`
 - Executar e modificar a app base
-

O que é o Node.js?

O **Node.js** é como o motor que corre aplicações escritas em JavaScript fora do browser — e o React precisa dele para funcionar!

 Instalação:

Vai a  <https://nodejs.org> e faz o download da versão LTS (Long Term Support).

 Confirma depois da instalação:

```
node -v  
npm -v
```

Instalar o Visual Studio Code (VS Code)

O VS Code é um editor moderno, leve e poderoso — o favorito dos programadores React!

 Instalação:

Vai a  <https://code.visualstudio.com> e instala a versão para o teu sistema operativo.

 Instala estas extensões recomendadas:

- ESLint
 - Prettier
 - React Developer Tools
-

Criar um Projeto com `create-react-app`

 Abre o terminal e escreve:

```
npx create-react-app minha-app-react
```

- Este comando cria uma pasta com todos os ficheiros base
- O npx vem incluído com o Node.js e executa ferramentas sem necessidade de instalação global

 Estrutura inicial:

```
minha-app-react/  
├── public/  
├── src/  
│   ├── App.js  
│   └── index.js  
└── package.json
```

Executar a App

No terminal:

```
cd minha-app-react  
npm start
```

 Vai abrir o browser em:

`http://localhost:3000`

E verás a famosa mensagem:

“Edit `src/App.js` and save to reload.”

Modificar a App Base

Abre o ficheiro `src/App.js` e muda o conteúdo do `return` para algo teu:

```
function App() {  
  return (  
    <div>  
      <h1>🚀 A minha primeira app React!</h1>  
      <p>Frontend em construção...</p>  
    </div>  
  );  
}
```

Grava e vê a magia acontecer ✨

Resumo Visual

Ferramenta	Função
Node.js + npm	Corre e gere pacotes JavaScript
VS Code	Editor moderno e eficiente

Ferramenta	Função
create-react-app	Cria estrutura base de projeto React
npm start	Arranca o servidor de desenvolvimento
App.js	Componente principal da tua app

Dica de Ouro

“No React, o `npm start` é como dar corda ao teu frontend. E o `App.js` é o palco principal do espetáculo! 🐱‍👤”

O que vem a seguir?

Na **Parte 8**, vais aprender a **usar o React para consumir a tua API RESTful** criada com Spring Boot. Vais ligar o frontend ao backend — é aqui que começa a comunicação entre mundos! 🌐 🔗

Parte 8 – Fundamentos de React: Componentes e JSX

React é como LEGO para a web: tudo é feito de blocos (componentes), encaixados com lógica simples mas poderosa!

Objetivos

- Compreender o que são **componentes**
 - Escrever componentes com **funções**
 - Usar **props** e **state**
 - Entender e aplicar **JSX**
 - Criar **componentes stateless**
 - Renderizar condicionalmente com **if** ou operadores ternários
-

1. O que são Componentes?

Um **componente React** é uma função que devolve JSX (uma espécie de HTML com superpoderes). Pensa nisto como uma **peça reutilizável da interface**.

```
function BemVindo() {  
  return <h1>Olá, React! 🙌</h1>;  
}
```

Dica:

Componentes devem começar com letra maiúscula: MeuComponente

2. JSX: O “HTML do React”

JSX é uma linguagem que parece HTML mas é escrita dentro do JavaScript. Permite misturar lógica e visual.

```
const nome = "Luís";  
const mensagem = <h2>Bem-vindo, {nome}!</h2>;
```

Regras de JSX:

- Um componente só pode devolver **um elemento principal** (envolve tudo num <div> ou <></>)
 - Usa **className** em vez de **class**
 - As expressões são colocadas entre **{ }**
-

3. Props: Dados que viajam até ao componente

“Props” são os **argumentos dos componentes** — como parâmetros de funções.

```
function Saudacao(props) {
  return <h2>Olá, {props.nome}!</h2>;
}
```

```
// Usar:
<Saudacao nome="Maria" />
```

4. State: Quando o componente precisa de memória

Usa `useState` para guardar valores que mudam com o tempo (como o contador de cliques).

```
import { useState } from "react";

function Contador() {
  const [conta, setConta] = useState(0);

  return (
    <div>
      <p>Valor atual: {conta}</p>
      <button onClick={() => setConta(conta + 1)}>+ Incrementar</button>
    </div>
  );
}
```

5. Renderização Condicional

Mostrar diferentes elementos com base em condições:

```
function BemVindo({ logado }) {
  return (
    <div>
      {logado ? <h2>Bem-vindo de volta!</h2> : <h2>Por favor, inicie sessão</h2>}
    </div>
  );
}
```

Também podes usar `if/else`, `&&` ou `?` : para lógica condicional

6. Componentes Stateless vs Stateful

- **Stateless:** não guardam dados, apenas mostram coisas com base nas props.
- **Stateful:** usam `useState` ou `useEffect` para reagir a alterações.

Exemplo stateless:

```
function MostraNome({ nome }) {
  return <p>O nome é {nome}</p>;
}
```

Resumo Visual

Conceito	Exemplo / Função React	Explicação
Componente	<code>function MeuComponente()</code>	Bloco reutilizável da UI
JSX	<code><h1>{variavel}</h1></code>	Mistura HTML + lógica JavaScript
Props	<code>props.nome</code>	Dados passados do pai
State	<code>useState(valorInicial)</code>	Guarda dados que mudam
Condicional	<code>condição ? A : B</code>	Mostra conteúdo baseado em lógica
Stateless	Só usa props	Sem <code>useState</code> , só recebe dados

Dica de Ouro

“No React, pensa pequeno: divide a interface em peças simples, como um puzzle. O segredo está em encaixar tudo com lógica clara e dados que fluem! 🧩🔗”

O que vem a seguir?

Na **Parte 9**, vais finalmente **ligar o React ao backend**, consumindo a tua API REST com `fetch` ou `axios`. A comunicação entre frontend e backend começa agora! 🌐🔗

Parte 9 – Hooks e Context API: React Moderno

Com os hooks, o React aprendeu truques novos! E com o Context, deixas de precisar de passar props como uma matrioska infinita!

Objetivos desta parte

- Compreender os hooks essenciais: `useState`, `useEffect`, `useRef`
 - Criar e usar **hooks personalizados**
 - Usar **Context API** para partilhar estado global entre componentes
-

1. `useState`: guardar estado num componente

```
import { useState } from "react";
```

```
function Contador() {  
  const [conta, setConta] = useState(0);  
  
  return (  
    <div>  
      <p>Valor atual: {conta}</p>  
      <button onClick={() => setConta(conta + 1)}>+ Incrementar</button>  
    </div>  
  );  
}
```

✦ A função `useState` devolve um array com:

1. O valor atual do estado
 2. Uma função para atualizá-lo
-

2. `useEffect`: efeitos colaterais (ex: chamadas API)

```
import { useEffect, useState } from "react";
```

```
function Dados() {  
  const [dados, setDados] = useState([]);  
  
  useEffect(() => {  
    fetch("https://jsonplaceholder.typicode.com/users")  
      .then(res => res.json())  
      .then(data => setDados(data));  
  }, []); // [] garante que só corre 1x no início  
  
  return (  
    <ul>  
      {dados.map(user => <li key={user.id}>{user.name}</li>)}  
    </ul>  
  );  
}
```



```

    </ul>
  );
}

```

⚠️ `useEffect` é como o `componentDidMount` das classes — ideal para carregar dados ou escutar eventos.

🔍 3. `useRef`: referência a elementos ou valores mutáveis

```

import { useRef } from "react";

function FocoInput() {
  const inputRef = useRef();

  function focar() {
    inputRef.current.focus();
  }

  return (
    <>
      <input ref={inputRef} />
      <button onClick={focar}> 🔴 Focar</button>
    </>
  );
}

```

🔧 `useRef` guarda um valor persistente entre renders, sem causar re-renderizações.

📁 4. Criar um **hook personalizado**

```

import { useState, useEffect } from "react";

function useHoraAtual() {
  const [hora, setHora] = useState(new Date().toLocaleTimeString());

  useEffect(() => {
    const timer = setInterval(() => {
      setHora(new Date().toLocaleTimeString());
    }, 1000);
    return () => clearInterval(timer);
  }, []);

  return hora;
}

```

Uso:

```

function Relogio() {
  const hora = useHoraAtual();

```

```
    return <p>🕒 Agora são {hora}</p>;  
  }
```

🌐 5. Context API: partilhar estado global

Sem Context:

```
<App>  
  <Header user={user}>  
    <Perfil user={user} />  
  </Header>  
</App>
```

Com Context:

```
<UserContext.Provider value={user}>  
  <Header />  
</UserContext.Provider>
```

Criar e usar Contexto:

```
import { createContext, useContext } from "react";
```

```
const UserContext = createContext();
```

```
function Header() {  
  const user = useContext(UserContext);  
  return <h2>👋 Olá, {user.nome}!</h2>;  
}
```

Uso:

```
<UserContext.Provider value={{ nome: "Luís" }}>  
  <Header />  
</UserContext.Provider>
```

🧠 Resumo Visual

Hook / API	Função	Equivalente clássico
useState	Guardar e alterar estado local	this.state / setState()
useEffect	Efeitos colaterais (API, timers, etc.)	componentDidMount
useRef	Guardar referência ou valor persistente	createRef, instance var
Custom Hook	Lógica reutilizável em vários componentes	—
Context + useContext	Estado global leve e partilhado	Redux simplificado

Dica de Ouro

“Com Hooks e Context, o React deixou de ser um mar de props e passou a ser um rio de dados fluido e controlado! 🚧💡”

A seguir...

Na **Parte 10**, vamos aprender a **consumir APIs REST com React**, usando fetch e axios, ligando finalmente o frontend ao backend numa experiência completa! 🔗🌐

🌀 Parte 10 – Eventos, Listas e Formulários em React

React não serve só para mostrar dados... serve para interagir! Aqui aprendes a reagir a cliques, gerir listas e criar formulários de forma elegante e controlada.

🎯 Objetivos

- Gerir **eventos** (como cliques, teclas, etc.)
 - Mostrar dados com **listas dinâmicas**
 - Criar **formulários controlados**
 - Entender **formulários não controlados** e quando usá-los
-

🎮 1. Gestão de Eventos

React usa funções inline ou métodos para responder a eventos como onClick, onChange, etc.

```
function Botao() {  
  const clicado = () => alert("Foste rápido no clique! 😊");  
  return <button onClick={clicado}>Clica aqui!</button>;  
}
```

Podes passar argumentos também:

```
<button onClick={() => dizerOla("Luís")}>Olá</button>;
```

📄 2. Mapeamento de Listas

Com map, podes criar componentes dinamicamente a partir de arrays.

```
const nomes = ["Ana", "Beatriz", "Carlos"];
```

```
function Lista() {  
  return (  
    <ul>  
      {nomes.map((nome, i) => (  
        <li key={i}>{nome}</li>  
      ))}  
    </ul>  
  );  
}
```

⚠ O atributo key é **obrigatório** para que o React consiga identificar cada item na lista.

3. Formulários Controlados

O React controla o valor do `<input>` através do `useState`. Isto permite ter controlo total sobre os dados introduzidos.

```
import { useState } from "react";

function Formulario() {
  const [nome, setNome] = useState("");

  const submeter = (e) => {
    e.preventDefault();
    alert(`Olá, ${nome}!`);
  };

  return (
    <form onSubmit={submeter}>
      <input
        type="text"
        value={nome}
        onChange={(e) => setNome(e.target.value)}
      />
      <button type="submit">Enviar</button>
    </form>
  );
}
```

4. Formulários Não Controlados

Se quiseses manipular diretamente o DOM (menos comum), usa `useRef`.

```
import { useRef } from "react";

function FormNaoControlado() {
  const inputRef = useRef();

  const submeter = (e) => {
    e.preventDefault();
    alert(`Valor: ${inputRef.current.value}`);
  };

  return (
    <form onSubmit={submeter}>
      <input type="text" ref={inputRef} />
      <button type="submit">Enviar</button>
    </form>
  );
}
```

Usar `useRef` é útil para inputs simples, mas perde-se o controlo do React sobre o valor.

Resumo Visual

Conceito	Exemplo	Explicação
Evento	<code>onClick, onChange</code>	Reação do utilizador
Lista	<code>array.map()</code>	Gera componentes com base em dados
Formulário Controlado	<code>value + onChange + useState</code>	React gere o input
Formulário Não Controlado	<code>useRef</code>	Acesso direto ao DOM

Dica de Ouro

“No React, os inputs são como conversas: ouves (`onChange`), respondes (`setState`) e controlas o rumo! 🎧 📧”

A seguir...

Na **Parte 11**, vais aprender a **ligar o React ao backend**: consumir APIs REST usando `fetch` e `axios`, populando as listas e formulários com dados reais! 🌐 ↻

Parte 11 – Consumir APIs REST com React

É nesta etapa que o frontend começa a "falar" com o backend — como dois amigos que trocam mensagens via HTTP!

Objetivos desta parte

- Entender o funcionamento de **promessas** e chamadas assíncronas
 - Usar `fetch` e `axios` para fazer pedidos à API
 - Apresentar dados vindos do backend (Spring Boot)
 - Tratar erros e atrasos nas respostas
-

1. Como funciona uma chamada API?

A tua aplicação React pode pedir dados a um servidor backend (por exemplo, Spring Boot) através de um **pedido HTTP** (GET, POST, PUT, DELETE, etc.).

Esse pedido devolve uma **promessa** (Promise) que será resolvida quando os dados chegarem.

2. Usar `fetch` (nativo do browser)

```
import { useEffect, useState } from "react";

function ListaCarros() {
  const [carros, setCarros] = useState([]);

  useEffect(() => {
    fetch("http://localhost:8080/api/cars")
      .then((res) => res.json())
      .then((data) => setCarros(data))
      .catch((erro) => console.error("Erro ao carregar dados:", erro));
  }, []);

  return (
    <ul>
      {carros.map((carro) => (
        <li key={carro.id}>{carro.brand} - {carro.model}</li>
      ))}
    </ul>
  );
}
```

💡 A função `useEffect` corre uma vez no início e vai buscar os dados à API.

3. Usar axios (mais poderoso)

Instala primeiro:

```
npm install axios
```

Código com axios:

```
import axios from "axios";
import { useEffect, useState } from "react";

function ListaCarros() {
  const [carros, setCarros] = useState([]);

  useEffect(() => {
    axios.get("http://localhost:8080/api/cars")
      .then((res) => setCarros(res.data))
      .catch((erro) => console.error("Erro:", erro));
  }, []);

  return (
    <ul>
      {carros.map((carro) => (
        <li key={carro.id}>{carro.brand} - {carro.model}</li>
      ))}
    </ul>
  );
}
```

✅ O axios tem uma sintaxe mais limpa e trata melhor os erros.

4. Promessas e Assíncrono

Uma função com async/await:

```
useEffect(() => {
  const carregarDados = async () => {
    try {
      const resposta = await axios.get("http://localhost:8080/api/cars");
      setCarros(resposta.data);
    } catch (erro) {
      console.error("Erro ao carregar:", erro);
    }
  };
  carregarDados();
}, []);
```

★ 5. POST, PUT e DELETE com axios

Criar (POST):

```
axios.post("http://localhost:8080/api/cars", {  
  brand: "Toyota",  
  model: "Corolla",  
});
```

Atualizar (PUT):

```
axios.put(`http://localhost:8080/api/cars/${id}`, dadosAtualizados);
```

Apagar (DELETE):

```
axios.delete(`http://localhost:8080/api/cars/${id}`);
```

Resumo Visual

Ação	Ferramenta	Método
Ler dados	fetch ou axios.get()	GET
Criar recurso	axios.post()	POST
Atualizar	axios.put()	PUT
Apagar	axios.delete()	DELETE

Dica de Ouro

“O React pergunta, o backend responde. Com axios ou fetch, estás sempre a trocar mensagens entre cliente e servidor! 🗉 🗉”

A seguir...

Na **Parte 12**, vamos aprender a usar **componentes de terceiros com React**, como o **React MUI** e **AG Grid**, para criar interfaces modernas e produtivas rapidamente! 🧩 🧩

Parte 12 – Componentes de Terceiros e MUI

O React brilha ainda mais quando usamos bibliotecas de componentes! Vais aprender a criar interfaces modernas, responsivas e altamente interativas com AG Grid, Material UI e React Router.

Objetivos

- Usar componentes externos como **AG Grid** e **Material UI (MUI)**
- Criar **interfaces modernas** com pouco código

- Adicionar **ícones interativos**
 - Implementar **navegação entre páginas** com React Router
-

1. AG Grid – Tabelas dinâmicas e poderosas

Instalar:

```
npm install ag-grid-react ag-grid-community
```

Exemplo de tabela:

```
import { AgGridReact } from "ag-grid-react";
import "ag-grid-community/styles/ag-grid.css";
import "ag-grid-community/styles/ag-theme-alpine.css";

const colunas = [
  { field: "marca" },
  { field: "modelo" },
  { field: "ano" },
];

const dados = [
  { marca: "Toyota", modelo: "Yaris", ano: 2020 },
  { marca: "BMW", modelo: "320i", ano: 2022 },
];

function TabelaCarros() {
  return (
    <div className="ag-theme-alpine" style={{ height: 300, width: 600 }}>
      <AgGridReact rowData={dados} columnDefs={colunas} />
    </div>
  );
}
```

⚡ Com poucas linhas, já tens uma tabela com filtros, ordenação e scroll automático!

2. Material UI – Interface com estilo

Instalar:

```
npm install @mui/material @emotion/react @emotion/styled
npm install @mui/icons-material
```

Botões e campos:

```
import { Button, TextField } from "@mui/material";

function Formulario() {
  return (
    <form>
      <TextField label="Nome" variant="outlined" fullWidth />
    </form>
  );
}
```

```

        <Button variant="contained" color="primary">Enviar</Button>
      </form>
    );
  }

```

💡 Ícones:

```
import DeleteIcon from "@mui/icons-material/Delete";
```

```
<Button startIcon={<DeleteIcon />}>Apagar</Button>
```

🧭 3. Navegação com React Router

📦 Instalar:

```
npm install react-router-dom
```

📄 Configurar rotas:

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
```

```

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">🏠 Home</Link> | <Link to="/carros">🚗 Carros</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/carros" element={<TabelaCarros />} />
      </Routes>
    </BrowserRouter>
  );
}

```

📊 Resumo Visual

Ferramenta	Função
AG Grid	Tabelas interativas e responsivas
MUI	Componentes visuais modernos e bonitos
MUI Icons	Ícones prontos para usar
React Router	Navegação entre páginas

💡 Dica de Ouro

“Com componentes de terceiros, a tua app deixa de parecer um rascunho e passa a parecer uma app profissional! 📁 ✨”

 SOON A seguir...

Na **Parte 13**, vais aprender a **testar o frontend com Jest e React Testing Library**, garantindo que tudo funciona mesmo quando há mudanças no código!  

Parte 13 – Integração Completa: Frontend para o Backend

Agora sim! Vamos unir as duas metades do nosso sistema: React no frontend e Spring Boot no backend, como uma ponte sólida entre o utilizador e os dados!

Objetivos desta parte

- Criar um mockup funcional da interface com React
 - Ligar o frontend ao backend com axios
 - Organizar a estrutura da app de forma limpa e escalável
-

1. Estrutura recomendada da aplicação integrada

Organiza o frontend React da seguinte forma:

/src	
├── components/	← Componentes reutilizáveis (ex: TabelaCarros)
├── pages/	← Páginas principais (ex: ListaCarros, AdicionarCarro)
├── services/	← Lógica para chamadas API
├── App.js	← Rotas e layout principal
└── index.js	← Entrada da aplicação

2. Mockup da interface (Ex: Lista de Carros)

Na pasta pages/ListaCarros.js:

```
import { useEffect, useState } from "react";
import axios from "axios";

function ListaCarros() {
  const [carros, setCarros] = useState([]);

  useEffect(() => {
    axios.get("http://localhost:8080/api/cars")
      .then(res => setCarros(res.data))
      .catch(erro => console.error("Erro ao buscar carros:", erro));
  }, []);

  return (
    <div>
      <h2>  Lista de Carros</h2>
      <table>
        <thead>
          <tr><th>Marca</th><th>Modelo</th></tr>
        </thead>
        <tbody>
```

```

        {carros.map(c => (
            <tr key={c.id}>
                <td>{c.brand}</td>
                <td>{c.model}</td>
            </tr>
        ))}
    </tbody>
</table>
</div>
);
}

```

```
export default ListaCarros;
```

🔗 3. Ligação à API Spring Boot

Cria um serviço em `services/CarroService.js`:

```

import axios from "axios";

const API_URL = "http://localhost:8080/api/cars";

export const listarCarros = () => axios.get(API_URL);
export const adicionarCarro = (carro) => axios.post(API_URL, carro);
export const apagarCarro = (id) => axios.delete(`${API_URL}/${id}`);
export const atualizarCarro = (id, carro) => axios.put(`${API_URL}/${id}`, carro);

```

Uso no componente:

```
import { listarCarros } from "../services/CarroService";
```

🌀 4. Adicionar Navegação com React Router

```

import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import ListaCarros from "../pages/ListaCarros";
import AdicionarCarro from "../pages/AdicionarCarro";

function App() {
    return (
        <BrowserRouter>
            <nav>
                <Link to="/">🏠 Home</Link> | <Link to="/carros">🚗 Carros</Link>
            </nav>
            <Routes>
                <Route path="/carros" element={<ListaCarros />} />
                <Route path="/carros/adicionar" element={<AdicionarCarro />} />
            </Routes>
        </BrowserRouter>
    );
}

```

```
}  
  
export default App;
```

Resumo Visual

Elemento	Função
/services/	Abstrai chamadas à API Spring Boot
axios	Biblioteca para consumo da API
useEffect	Executa carregamento inicial de dados
React Router	Permite navegação SPA (Single Page App)
Componentes por página	Organização clara por funcionalidades

Dica de Ouro

“Unir React ao Spring Boot é como criar um ecossistema completo: front e back a trabalhar em harmonia! 🌱 📱”

A seguir...

Na **Parte 14**, vais adicionar **CRUD completo (Create, Read, Update, Delete)** no frontend com formulários e chamadas API — e vais ver a tua aplicação a ganhar corpo! 🏗️ ✨

Parte 14 – CRUD Completo com React + Spring Boot

CRUD: Create, Read, Update, Delete – os 4 pilares de qualquer aplicação que manipule dados. Vamos fazê-los todos de forma clara, moderna e visual.

Objetivos desta parte

- Implementar **listagem, adição, edição e eliminação** de dados
 - Ligar tudo ao backend com axios
 - Mostrar **feedback visual** (ex: alertas, confirmações)
 - Manter o código limpo e modular com boas práticas
-

Estrutura Recomendada

```
/src
├── components/
│   ├── ListaCarros.js
│   └── FormularioCarro.js
├── pages/
│   └── Carros.js
└── services/
    └── CarroService.js
```

1. Listar Registos (Read)

```
// pages/Carros.js
import { useEffect, useState } from "react";
import { listarCarros, apagarCarro } from "../services/CarroService";

function Carros() {
  const [carros, setCarros] = useState([]);

  useEffect(() => {
    listarCarros().then((res) => setCarros(res.data));
  }, []);

  const eliminar = (id) => {
    if (window.confirm("Tem a certeza?")) {
      apagarCarro(id).then(() => {
        setCarros(carros.filter((c) => c.id !== id));
        alert("Registo eliminado com sucesso.");
      });
    }
  };

  return (
```



```

<div>
  <h2> 🚗 Lista de Carros</h2>
  <table>
    <thead>
      <tr><th>Marca</th><th>Modelo</th><th>Ações</th></tr>
    </thead>
    <tbody>
      {carros.map((carro) => (
        <tr key={carro.id}>
          <td>{carro.brand}</td>
          <td>{carro.model}</td>
          <td>
            <button onClick={() => eliminar(carro.id)}> 🗑 Apagar</button>
          </td>
        </tr>
      ))}
    </tbody>
  </table>
</div>
);
}

```

+ 2. Adicionar Registos (Create)

```

// components/FormularioCarro.js
import { useState } from "react";
import { adicionarCarro } from "../services/CarroService";

export default function FormularioCarro() {
  const [carro, setCarro] = useState({ brand: "", model: "" });

  const handleSubmit = (e) => {
    e.preventDefault();
    adicionarCarro(carro).then(() => {
      alert("Carro adicionado com sucesso!");
      setCarro({ brand: "", model: "" });
    });
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        placeholder="Marca"
        value={carro.brand}
        onChange={(e) => setCarro({ ...carro, brand: e.target.value })}
      />
      <input
        placeholder="Modelo"
        value={carro.model}
        onChange={(e) => setCarro({ ...carro, model: e.target.value })}
      />
    </form>
  );
}

```

```

    <button type="submit">📁 Guardar</button>
  </form>
);
}

```

3. Atualizar Registos (Update)

Podes reutilizar o mesmo formulário, mas pré-preenchido com os dados do carro selecionado, e enviar via `axios.put(...)`. Aqui está um exemplo base:

```

export const atualizarCarro = (id, carro) =>
  axios.put(`${API_URL}/${id}`, carro);

```

No frontend, preenches os campos com `useEffect` e atualizas:

```

useEffect(() => {
  axios.get(`${API_URL}/${id}`).then((res) => setCarro(res.data));
}, [id]);

```

4. Serviço API (CarroService.js)

```

import axios from "axios";
const API_URL = "http://localhost:8080/api/cars";

export const listarCarros = () => axios.get(API_URL);
export const adicionarCarro = (carro) => axios.post(API_URL, carro);
export const apagarCarro = (id) => axios.delete(`${API_URL}/${id}`);
export const atualizarCarro = (id, carro) => axios.put(`${API_URL}/${id}`, carro);

```

5. Feedback Visual

Usa `alert()`, `window.confirm()` ou bibliotecas como `react-toastify` para mensagens mais amigáveis.

```
npm install react-toastify
```

Resumo CRUD

Operação	Método HTTP	Função Axios	Interface
Criar	POST	<code>adicionarCarro</code>	Formulário
Ler	GET	<code>listarCarros</code>	Tabela/Listagem
Atualizar	PUT	<code>atualizarCarro</code>	Formulário com dados
Apagar	DELETE	<code>apagarCarro</code>	Botão 🗑️

Dica de Ouro

"**CRUD** é o alfabeto de uma aplicação dinâmica: com ele, podes construir frases, histórias... ou sistemas completos! 📄 🖥️"

A seguir...

Na **Parte 15**, vais aprender a aplicar **estilo com Material UI (MUI)** em todos os teus componentes — tornando o frontend elegante, responsivo e profissional. ✨ 🎨

Parte 15 – Estilização Profissional com MUI

O conteúdo já está funcional — agora é hora de o tornar visualmente apelativo, acessível e profissional com MUI.

Objetivos

- Utilizar os principais **componentes visuais do MUI**
 - Criar e aplicar **temas personalizados**
 - Melhorar a **UX** com feedback visual claro
 - Estilizar botões, campos (TextField) e notificações
-

1. Instalação do Material UI

```
npm install @mui/material @emotion/react @emotion/styled
npm install @mui/icons-material
```

2. Usar Botões com Estilo

```
import { Button } from '@mui/material';
```

```
<Button variant="contained" color="primary">Guardar</Button>
<Button variant="outlined" color="secondary">Cancelar</Button>
<Button variant="text" color="error">Eliminar</Button>
```

 Dica: variant="contained" dá um look sólido; outlined é mais leve.

3. Formulários com TextField

```
import { TextField } from '@mui/material';
```

```
<TextField
  label="Marca"
  variant="outlined"
  fullWidth
  margin="normal"
/>
<TextField
  label="Modelo"
  variant="outlined"
  required
/>
```

 UX melhorada: mensagens de erro, sugestões e preenchimento guiado.

🧠 4. Criar um Tema Personalizado

```
import { createTheme, ThemeProvider } from '@mui/material/styles';

const tema = createTheme({
  palette: {
    primary: { main: '#1976d2' },
    secondary: { main: '#9c27b0' },
  },
  shape: {
    borderRadius: 12,
  },
});

function App() {
  return (
    <ThemeProvider theme={tema}>
      {/* resto da app */}
    </ThemeProvider>
  );
}
```

🍬 Com o ThemeProvider, todo o visual da aplicação fica coeso.

🔔 5. Feedback com Snackbar (avisos rápidos)

```
import { Snackbar, Alert } from '@mui/material';

<Snackbar open={aberto} autoHideDuration={3000} onClose={fechar}>
  <Alert onClose={fechar} severity="success" sx={{ width: '100%' }}>
    Carro adicionado com sucesso!
  </Alert>
</Snackbar>
```

🌟 6. Ícones com @mui/icons-material

```
import DeleteIcon from '@mui/icons-material/Delete';

<Button startIcon={<DeleteIcon />} color="error">Eliminar</Button>
```

Ícones dão **clareza e intuição** às ações do utilizador 🗨️

💡 Boas Práticas de UX

Elemento	Boa Prática UX
Botões	Cores e ícones coerentes com a ação
Campos de texto	Labels claros, validação imediata

Elemento	Boa Prática UX
Feedback	Snackbar para confirmar ações
Responsividade	Grid, Box, Stack para layout adaptável

Dica de Ouro

"O utilizador sente confiança numa app bonita, clara e bem organizada. UX e UI não são extra — são parte do valor! 🌟"

A seguir...

Na **Parte 16**, vamos aprender a fazer **testes de frontend com Jest e React Testing Library**, garantindo que o que criámos com tanto cuidado funciona sempre como esperado! 🧪✅

Parte 16 – Testes Automatizados no Frontend com Jest

Numa aplicação profissional, testar é tão importante como programar. Com Jest e React Testing Library, vais aprender a testar interações, renderizações e até capturar regressões com snapshots.

Objetivos desta parte

- Compreender os fundamentos do **Jest** e da **React Testing Library**
 - Criar testes para **componentes, eventos e estados**
 - Utilizar **snapshot testing** para detetar alterações inesperadas no UI
-

1. Instalação do ambiente de testes

Se usaste create-react-app, já tens Jest e @testing-library/react incluídos.

Caso contrário, instala com:

```
npm install --save-dev jest @testing-library/react @testing-library/jest-dom
```

2. Testes de componentes básicos

Suponhamos que temos este componente:

```
function Saudacao({ nome }) {  
  return <h1>Olá, {nome}!</h1>;  
}
```

Teste simples:

```
import { render, screen } from '@testing-library/react';  
import Saudacao from './Saudacao';  
  
test('exibe saudação com o nome', () => {  
  render(<Saudacao nome="Luís" />);  
  expect(screen.getByText(/Olá, Luís!/i)).toBeInTheDocument();  
});
```

3. Testar eventos (ex: cliques)

```
function BotaoContador() {  
  const [contador, setContador] = useState(0);  
  return (  
    <div>  
      <p>Contador: {contador}</p>  
      <button onClick={() => setContador(contador + 1)}>Incrementar</button>  
    </div>  
  );  
}
```

```
);  
}
```

☑ Teste de clique:

```
import { render, screen, fireEvent } from '@testing-library/react';  
import BotaoContador from './BotaoContador';  
  
test('incrementa o contador ao clicar', () => {  
  render(<BotaoContador />);  
  fireEvent.click(screen.getByText('Incrementar'));  
  expect(screen.getByText(/Contador: 1/i)).toBeInTheDocument();  
});
```

🧩 4. Snapshot Testing

Ideal para componentes que mudam visualmente:

```
import { render } from '@testing-library/react';  
import Saudacao from './Saudacao';  
  
test('snapshot do componente Saudacao', () => {  
  const { asFragment } = render(<Saudacao nome="Maria" />);  
  expect(asFragment()).toMatchSnapshot();  
});
```

Ao correr o teste pela primeira vez, o Jest vai guardar um "snapshot" do HTML renderizado. Se mudares algo no componente, o teste falha — ajudando a detetar mudanças visuais não intencionais.

🔄 5. Executar os testes

`npm test`

Usa `u` para atualizar snapshots ou `a` para correr todos os testes.

📊 Resumo Visual

Tipo de Teste	Biblioteca	Exemplo
Renderização	@testing-library/react	<code>render(<Comp />)</code>
Interação (click, etc)	<code>fireEvent</code> ou <code>userEvent</code>	<code>fireEvent.click(...)</code>
Estado visual (UI)	snapshot testing	<code>expect(asFragment()).toMatchSnapshot()</code>

💡 Dica de Ouro

"Testes automatizados são como seguros: podes não usá-los todos os dias, mas um dia vais agradecer por tê-los! 🌈 ✅"

 SOON A seguir...

Na **Parte 17**, vamos explorar a **Segurança no Frontend com JWT** — para proteger rotas, gerir sessões e lidar com tokens de forma prática!  

Parte 17 – Segurança Full Stack com JWT

A tua aplicação está funcional, bonita e testada... agora precisa de segurança! Nesta parte, vamos proteger tanto o backend como o frontend com JWT.

Objetivos desta parte

- Proteger endpoints do backend com **Spring Security + JWT**
 - Gerar e validar **tokens JWT**
 - Criar **login seguro**
 - Proteger rotas no **frontend React**
-

1. Backend Seguro com Spring Security e JWT

a) Adicionar dependências no pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt</artifactId>
  <version>0.9.1</version>
</dependency>
```

b) Criar classes essenciais

- User.java → Entidade com username e password
 - UserRepository.java → Interface para aceder aos utilizadores
 - JwtUtil.java → Gera e valida tokens JWT
 - JwtFilter.java → Intercepta pedidos e valida tokens
 - SecurityConfig.java → Configuração do Spring Security
-

c) Exemplo: Gerar token após login

```
@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody AuthRequest authRequest) {
    Authentication auth = authManager.authenticate(
        new UsernamePasswordAuthenticationToken(
            authRequest.getUsername(), authRequest.getPassword()));

    final String token = jwtUtil.generateToken(authRequest.getUsername());
    return ResponseEntity.ok(new AuthResponse(token));
}
```

d) Proteger endpoints REST

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http.csrf().disable()
        .authorizeRequests()
        .antMatchers("/login").permitAll()
        .anyRequest().authenticated()
        .and()
        .sessionManagement()
        .sessionCreationPolicy(SessionCreationPolicy.STATELESS);

    http.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
}
```

2. Frontend: Gestão do Token com React

a) Guardar token no localStorage

```
localStorage.setItem("token", data.token);
```

b) Enviar token no cabeçalho com axios

```
axios.get("http://localhost:8080/api/cars", {
  headers: {
    Authorization: `Bearer ${localStorage.getItem("token")}`,
  }
});
```

c) Proteger rotas com um wrapper de autenticação

```
import { Navigate } from "react-router-dom";

function RotaProtegida({ children }) {
  const token = localStorage.getItem("token");
  return token ? children : <Navigate to="/login" />;
}
```

Uso no App.js:

```
<Route path="/carros" element={<RotaProtegida><ListaCarros /></RotaProtegida>} />
```

🔮 d) Exemplo de formulário de login

```
import { useState } from "react";
import axios from "axios";

function Login() {
  const [credenciais, setCredenciais] = useState({ username: "", password: "" });

  const login = () => {
    axios.post("http://localhost:8080/login", credenciais)
      .then(res => {
        localStorage.setItem("token", res.data.token);
        alert("Login feito com sucesso!");
        // redirecionar...
      }).catch(() => alert("Credenciais inválidas"));
  };

  return (
    <div>
      <input placeholder="Utilizador" onChange={(e) => setCredenciais({ ...credenciais,
username: e.target.value })} />
      <input type="password" placeholder="Password" onChange={(e) => setCredenciais({
...credenciais, password: e.target.value })} />
      <button onClick={login}>🔑 Entrar</button>
    </div>
  );
}
```

📊 Resumo Visual

Componente	Função
JWT	Gera token seguro
Spring Security	Protege endpoints REST
Axios com token	Garante chamadas autenticadas
React Router	Protege rotas privadas com Navigate

💡 Dica de Ouro

"Com JWT, cada utilizador recebe uma chave que lhe dá acesso ao castelo — mas essa chave expira! 🏰 🔑"

➡️ A seguir...

Na **Parte 18**, vais aprender a **fazer o deploy da aplicação completa**, com backend e frontend prontos para o mundo real, usando Docker e serviços de cloud! 🚀🌐

Parte 18 – Deploy Profissional com Docker e Heroku

Transforma a tua app local numa aplicação publicada e acessível, usando Docker e Heroku (ou outro serviço cloud). Aqui aprendemos a empacotar tudo com Docker e configurar variáveis de ambiente.

Objetivos

- Criar imagens Docker para o backend e frontend
 - Usar Dockerfile e docker-compose
 - Configurar variáveis de ambiente (.env)
 - Fazer deploy na cloud (ex: Heroku, Render, Fly.io)
-

1. Criar Dockerfile para o Backend

No projeto Spring Boot (cardatabase):

Dockerfile:

```
FROM openjdk:17-jdk-slim
VOLUME /tmp
ARG JAR_FILE=target/*.jar
COPY ${JAR_FILE} app.jar
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

 Construir a imagem:

```
mvn clean package
docker build -t cardatabase-backend .
```

2. Criar Dockerfile para o Frontend (React)

No projeto React:

Dockerfile:

```
FROM node:18 as build
WORKDIR /app
COPY . .
RUN npm install && npm run build

FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
```

 Construir a imagem:

```
docker build -t cardatabase-frontend .
```



3. Testar com docker-compose

Cria o ficheiro `docker-compose.yml`:

```
version: '3'
services:
  backend:
    build: ./backend
    ports:
      - "8080:8080"
    environment:
      - SPRING_DATASOURCE_URL=jdbc:mysql://db:3306/cardb
      - SPRING_DATASOURCE_USERNAME=root
      - SPRING_DATASOURCE_PASSWORD=root
    depends_on:
      - db

  frontend:
    build: ./frontend
    ports:
      - "3000:80"

  db:
    image: mariadb
    environment:
      - MYSQL_ROOT_PASSWORD=root
      - MYSQL_DATABASE=cardb
```



Correr tudo:

`docker-compose up`



4. Deploy na Cloud

a) Backend no Heroku (alternativa: Render)

1. Criar conta em heroku.com
2. Instalar o CLI:

`npm install -g heroku`

3. Fazer login:

`heroku login`

4. Criar app:

`heroku create minha-api-cardatabase`

5. Push do projeto (usar `heroku-deploy` ou GitHub Actions)

Alternativas modernas:

- [Render.com](https://render.com)
- [Fly.io](https://fly.io)

- [Railway.app](#)
-

5. Variáveis de Ambiente com .env

No React:

```
REACT_APP_API_URL=https://minha-api.com/api
```

Usar:

```
const url = process.env.REACT_APP_API_URL;
```

No Spring Boot:

```
spring.datasource.url=${SPRING_DATASOURCE_URL}
```

6. Segurança e Produção

- React deve usar HTTPS (podes configurar com NGINX ou Vite)
 - Tokens JWT devem ser armazenados com cuidado (localStorage ou cookies HTTPOnly)
 - Usa .dockerignore e .gitignore para evitar ficheiros sensíveis
-

Dica de Ouro

"Fazer deploy é como lançar uma ponte entre o teu computador e o mundo. Faz com confiança e segurança! 🧠🔒"

A seguir...

Na **Parte 19**, vamos aplicar **boas práticas de desenvolvimento**, como versionamento com Git, organização de projeto e guidelines para equipas. 🧠📁

Parte 19 – Melhores Práticas para Devs Full Stack

Tecnologia é importante, mas o que te torna um verdadeiro profissional são os hábitos. Aqui estão as melhores práticas para brilhar como Dev Full Stack!

Objetivos

- Aplicar boas práticas com **Git**
 - Escrever APIs REST de forma elegante e escalável
 - Usar **convenções de código limpas**
 - Criar **interfaces responsivas**
 - Pensar como equipa (mesmo quando se trabalha sozinho)
-

1. Versionamento com Git

Boas práticas:

-  **Commits pequenos e frequentes**
-  Nomes claros e no imperativo: adiciona funcionalidade de login
-  Usar **branches**:
 - main ou master → versão estável
 - dev → desenvolvimento contínuo
 - feature/nome → funcionalidades específicas

Exemplo de fluxo:

```
git checkout -b feature/login
# trabalha...
git add .
git commit -m "adiciona formulário de login"
git push origin feature/login
```

2. Boas práticas para APIs REST

Princípio	Exemplo
 Verbo correto	GET /api/cars, POST /api/cars
 Rotas descritivas	/api/cars em vez de /api/getAllCars
 Não misturar verbos no nome	Evitar /deleteCar — usa DELETE /car/1
 Respostas com status	200 OK, 201 Created, 404 Not Found
 Separação clara de entidades	/users, /cars, /owners

3. Convenções de código (backend e frontend)

Java (Spring Boot):

- Classes: CamelCase → CarRepository, JwtFilter
- Variáveis: camelCase → userRepository
- Constantes: MAIUSCULAS_COM_UNDERSCORES
- Separar lógica em Controller, Service e Repository

JavaScript / React:

- Componentes: CamelCase → ListaCarros.js
- Funções: nomes claros → handleClick, loadCars
- Evitar lógica duplicada: usa funções auxiliares
- Comentários só quando **acrescentam valor**

4. Design Responsivo e Mobile-First

Boas práticas:

- Usa Grid e Flexbox para adaptação fluida
- Testa em tamanhos de ecrã diferentes
- Usa media queries ou componentes do MUI responsivos:

```
<Grid container spacing={2}>  
  <Grid item xs={12} md={6}>  
    <TextField fullWidth label="Marca" />  
  </Grid>  
</Grid>
```

 xs={12} → ocupa tudo em ecrãs pequenos
md={6} → ocupa metade em médios/grandes

5. Mentalidade de Profissional Full Stack

-  Mantém a estrutura do projeto limpa e modular
-  Escreve testes unitários (mesmo que simples!)
-  Documenta rotas e exemplos no Postman
-  Automatiza tarefas (build, deploy, testes)
-  Aprende a trabalhar com equipas e git colaborativo (PRs, revisões)

Resumo Visual

Prática	Ferramenta / Exemplo
Versionamento	Git, branches, commits claros
API REST limpa	Verbos HTTP + rotas descritivas

Prática	Ferramenta / Exemplo
Convenção de código	CamelCase, nomes claros
Design responsivo	Grid, breakpoints, mobile-first
Organização de projeto	Componentização + separação de responsabilidades

Dica de Ouro

"Código limpo é como uma boa conversa: clara, direta e sem ruído. Profissionalismo não é saber tudo — é saber trabalhar bem com os outros e com o futuro tu. 🧑💻"

A seguir...

Na **Parte 20**, vamos concluir o tutorial com um mini-projeto completo que junta **frontend, backend, segurança, deploy e boas práticas** — uma simulação realista de um sistema Full Stack! 🎯🎓



Parte 20 – Conclusão e Caminhos para o Futuro

O projeto está feito, testado, seguro e no ar. Mas o mundo da tecnologia nunca pára. Aqui está o teu mapa para continuar a evoluir.



Objetivos

- Apresentar **tecnologias complementares** ao stack atual
- Explorar **GraphQL**, testes **E2E** e **CI/CD**
- Definir um **roadmap de evolução full stack**



1. Tecnologias Complementares para o Stack

Área	Alternativas ou Complementos
Base de dados	PostgreSQL, MongoDB, Firebase
Backend Java	Micronaut, Quarkus, Jakarta EE
Frontend	Next.js (SSR), Vue.js, Angular
Mobile	React Native, Flutter, Kotlin
Cloud	AWS, Azure, GCP, Railway, Render
Segurança	OAuth2, Auth0, Keycloak



2. GraphQL – A evolução das APIs

REST: múltiplos endpoints (ex: /carros, /utilizadores, etc.)

GraphQL: um único endpoint onde pedes exatamente os dados que precisas.

```
query {  
  carros {  
    marca  
    preco  
  }  
}
```



Benefícios:

- Menos chamadas
- Mais controlo do lado do frontend
- Documentação automática (GraphiQL)



Requer setup adicional no backend (spring-boot-starter-graphql) e no frontend (apollo-client).

3. Testes End-to-End (E2E)

Testes E2E garantem que **toda a app funciona como esperado**, do frontend ao backend.

Ferramentas recomendadas:

- **Cypress**: intuitiva e moderna
- **Playwright**: poderosa e multiplataforma

Exemplo com Cypress:

```
describe("Login", () => {
  it("permite login com credenciais válidas", () => {
    cy.visit("/login");
    cy.get("input[name=username]").type("admin");
    cy.get("input[name=password]").type("1234");
    cy.get("button").click();
    cy.url().should("include", "/dashboard");
  });
});
```

4. Integração Contínua e Deploy Contínuo (CI/CD)

Com CI/CD, cada alteração no código pode ser automaticamente:

1. Testada 
2. Compilada 
3. Empacotada 
4. Publicada na cloud 

Exemplos de ferramentas:

- **GitHub Actions** (nativo do GitHub)
- **GitLab CI/CD**
- **CircleCI**
- **Jenkins**

Exemplo de GitHub Actions para React:

```
name: Deploy React App
on:
  push:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: npm install
      - run: npm run build
```

5. Roadmap Full Stack – O Teu Próximo Nível

Fundamentos Sólidos

- HTML, CSS, JavaScript moderno (ES6+)
- Git e GitHub
- Testes unitários e E2E

Backend Avançado

- Spring Security com OAuth2
- Spring WebFlux (reativo)
- Microserviços com Spring Cloud

Frontend Avançado

- React com Redux, Zustand ou Recoil
- SSR com Next.js
- Design Systems e Storybook

DevOps

- Docker avançado
- Kubernetes (K8s)
- Terraform e infraestrutura como código

Soft Skills e Carreira

- Clean Code e Design Patterns
- Comunicação e trabalho em equipa
- Contribuição em open-source

Dica Final

“Tornar-se um Dev Full Stack não é dominar tudo, mas sim saber navegar em várias áreas com confiança e curiosidade.”  